

Assignment - Python_Django Developer _V1 (2).docx

You are an experienced Python/Django Developer. I have uploaded my Django hiring assignment. Read the entire assignment carefully before writing any code. Do not skip or ignore any requirement mentioned in the assignment. I want this project to look like it was developed by a Junior Python/Django Developer. Important Instructions: - Keep the code simple, clean, readable, and well-organized. - Do NOT over-engineer the solution. - Use only Python, Django, Django REST Framework, and SQLite. - Do NOT use Docker, Redis, Celery, JWT, async programming, microservices, PostgreSQL, or any advanced architecture unless the assignment explicitly requires it. - Follow PEP 8 coding standards. - Add meaningful comments only where necessary. - Use simple and understandable variable and function names. - The project should be easy to explain in a technical interview. Before writing any code: 1. Read the complete assignment. 2. Explain all requirements. 3. List every task that needs to be completed. 4. Design the database. 5. Explain the API endpoints. 6. Show the project folder structure. 7. Explain the box recommendation algorithm. 8. Mention edge cases and validations. Wait for my approval before starting implementation. After I approve: Build the complete project step by step. Include: - Django Project - Django App - Models - Admin Panel - Serializers - API Views - URL Configuration - Business Logic - Input Validation - Error Handling - Unit Tests - requirements.txt - README.md with installation and API usage instructions After the project is complete: 1. Review the entire project. 2. Fix every bug. 3. Verify that every assignment requirement is completed. 4. Make sure the project runs without errors. 5. Ensure all unit tests pass. 6. Remove unnecessary code. 7. Keep the project simple and interview-friendly. 8. Package the complete project into a ZIP file. 9. Provide the ZIP file for download. Do not stop until the entire project is completed successfully.

Show less

Thursday 3:41 PM

Worked for 2m 46s

I read the complete .docx. It contains a short open-ended assignment, not a detailed API spec, so we need to design a sensible simple Django/DRF solution around the stated goal.

Assignment Requirements

Build a small Django system for an ecommerce warehouse that recommends the most suitable shipping box for an order.

The system must consider:

- Product dimensions
- Product weight
- Box internal dimensions
- Box maximum weight capacity
- Box cost
- Recommendation of the most suitable box for an order

Submission must include:

- GitHub repository link
- README.md
- AI_USAGE.md
- Exported real chat transcript, not AI-generated
- A personal “What did you learn?” answer, not AI-generated
- Test cases
- Test run output via GitHub Actions or TEST_OUTPUT.md

Important note: I should not generate the “What did you learn?” answer for you. I can leave a clearly marked placeholder file or README section for you to fill manually.

Tasks To Complete After Approval

1. Create Django project and app.
2. Configure SQLite and Django REST Framework.
3. Create models for products, boxes, orders, and order items.
4. Register models in Django admin.
5. Create serializers with validation.
6. Create API views/endpoints.
7. Add box recommendation business logic.
8. Add input validation and useful error responses.
9. Add unit tests for models, APIs, and recommendation logic.
10. Add requirements.txt.
11. Add README.md with setup and API usage.
12. Add AI_USAGE.md.
13. Add TEST_OUTPUT.md after tests run.
14. Add a placeholder for your personal learning answer.
15. Run and fix tests.
16. Package the full project as a ZIP.

Database Design

Simple and interview-friendly:

- Product
 - name
 - length
 - width

- height
 - weight
- Box
 - name
 - inner_length
 - inner_width
 - inner_height
 - max_weight
 - cost
 - is_active
- Order
 - customer_name
 - created_at
- OrderItem
 - order
 - product
 - quantity

No advanced architecture. No PostgreSQL, Docker, Redis, Celery, JWT, or async.

API Endpoints

Proposed simple REST endpoints:

- GET /api/products/
- POST /api/products/
- GET /api/products/<id>/
- PUT /api/products/<id>/
- DELETE /api/products/<id>/
- GET /api/boxes/
- POST /api/boxes/
- GET /api/boxes/<id>/
- PUT /api/boxes/<id>/
- DELETE /api/boxes/<id>/
- GET /api/orders/

- POST /api/orders/
- GET /api/orders/<id>/
- GET /api/orders/<id>/recommend-box/

The recommendation endpoint returns the best box or a clear error if no box fits.

Project Structure

box_selection_project/

manage.py

requirements.txt

README.md

AI_USAGE.md

TEST_OUTPUT.md

LEARNING.md

box_selection_project/

__init__.py

settings.py

urls.py

wsgi.py

asgi.py

shipping/

__init__.py

admin.py

apps.py

models.py

serializers.py

views.py

urls.py

services.py

tests.py

migrations/

Box Recommendation Algorithm

For each order:

1. Calculate total order weight.
2. Calculate required product dimensions.
3. Check every active box.
4. A box is valid only if:
 - total weight $\leq$ box max weight
 - products fit inside the box dimensions
5. Sort valid boxes by:
 - lowest cost first
 - then smallest unused volume, as a tie-breaker
6. Return the best box.

To keep this junior-friendly, I'll use a simple bounding-box approach:

- For a single product, check if its dimensions fit in the box in any rotation.
- For multiple products, estimate total packed volume and max dimensions.
- This is not a perfect 3D packing engine, but it is simple, explainable, and suitable for a hiring assignment unless exact 3D bin packing is explicitly required.

Edge Cases And Validations

I will handle:

- Product dimensions must be positive.
- Product weight must be positive.
- Box dimensions must be positive.
- Box max weight must be positive.
- Box cost cannot be negative.
- Order must contain at least one item.
- Item quantity must be positive.
- Inactive boxes should not be recommended.
- If no box fits, return a clear 404 style response.
- If order ID does not exist, return 404.
- If product is too large or too heavy, return a useful message.
- If multiple boxes fit, choose cheapest.
- If same cost, choose better space efficiency.

I'm ready to start implementation after your approval.

Thursday 3:44 PM

Approved. Please keep the implementation simple and junior-friendly. Do not over-engineer the solution. Use only basic Django and Django REST Framework concepts. Do not create LEARNING.md. I will write my own learning answer. Use a simple recommendation algorithm based on dimensions, weight, and lowest box cost. Avoid complex packing or optimization algorithms. After completing the project: - Review the entire code. - Fix all bugs. - Ensure all assignment requirements are satisfied. - Run all tests. - Create a ZIP file containing the complete project.

Thursday 3:45 PM

Worked for 29m 51s

Done. I built the Django + DRF project, reviewed it, fixed issues, ran migrations, ran tests, and packaged the clean deliverable ZIP.

Final ZIP: box_selection_project_final.zip

Verified:

- python manage.py check passes
- python manage.py migrate passes
- python manage.py test passes: 9 tests OK
- ZIP excludes .venv, SQLite files, cache files, and the unrelated old sales files
- No LEARNING.md was created, as requested